Chapter 7. AutoML and AutoGluon

Predictive Modeling, Lecture 6 (model selection, OOF predictions, fold bagging, stacking, weighted ensembles)

Contents

1 When There Are Many Candidates, Model Selection Becomes a Problem **2**

2 Stacking: Candidate Predictions Become New Inputs **2**

2.1 Why the level-1 inputs must be out-of-fold 2

3 K-fold Bagging and OOF Predictions **2**

4 Weighted Ensembles **3**

5 How the Three Mechanisms Fit Together in AutoGluon **4**

6 Setting the Search Space **4**

7 Selected exercises **5**

1 When There Are Many Candidates, Model Selection Becomes a Problem

No tabular model is always best, so the question is *how to combine the candidates* of Lectures 1–5.

Stage	What AutoGluon does
fit candidates	trains several candidates: linear models, tree boosting, neural networks, ...
internal validation	non-bagged models on a holdout set, bagged models with OOF predictions
ensemble	combines candidates by fold bagging, stacking, weighted ensembles
leaderboard	compares internal validation score, fit time, prediction time, stack level
final prediction	predicts new cases with the selected single model or ensemble

AutoGluon **manages fitting, validation and combination**; its internal splits are random/stratified, so time/group structure and a protected test set must be imposed from outside (Ch. 1).

2 Stacking: Candidate Predictions Become New Inputs

Candidate predictions for x_i:

$$z_i = (h_{\text{linear}}(x_i),\ h_{\text{boost}}(x_i),\ h_{\text{MLP}}(x_i)), \qquad \text{e.g. } z_i = (0.30,\ 0.75,\ 0.60)$$

(probabilities) become features of a higher-level model:

$$\hat{y}_i = g(x_i, z_i),$$

Stacking **learns again from the candidates' predictions**, exploiting that they err on different cases.

2.1 Why the level-1 inputs must be out-of-fold

Theorem: in-sample stacking rewards overfitting

If g is trained on in-sample predictions $z_i = h(x_i)$ of models that were fitted on the same rows, a model that memorizes the training data (e.g. 1-nearest neighbor, whose in-sample prediction is exactly y_i) looks perfect, and g puts all its weight on it, although its out-of-sample error can be the worst of all candidates.

g must train on predictions for cases *the candidate did not see*, as at prediction time: OOF predictions.

3 K-fold Bagging and OOF Predictions

Definition: fold bagging

Split the training data into folds $F_1, \ldots, F_K$ and fit the same model K times, the kth time without F_k, giving $h^{(-1)}, \ldots, h^{(-K)}$ (same algorithm and settings, different training rows).

- **Training data: OOF predictions.** $i \in F_k \implies \hat{y}_i^{\text{OOF}} = h^{(-k)}(x_i)$: each case is predicted by the model that did not use it.
- **New cases: averaged prediction.** $x_{\text{new}} \mapsto \frac{1}{K}\sum_{k=1}^{K} h^{(-k)}(x_{\text{new}})$.

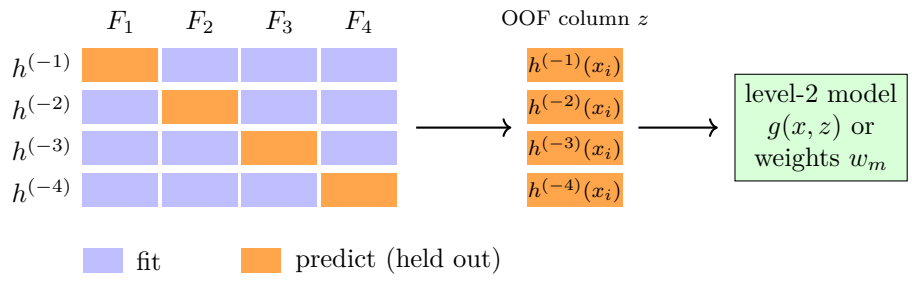


Figure 1: Fold bagging, $K = 4$: $h^{(-k)}$ predicts only F_k, giving an OOF column z on which the level-2 learner is trained.

The OOF vector gives (i) an honest internal score and (ii) leakage-free stacking inputs; averaging fold models lowers variance (Ch. 4). `num_bag_sets` repeats with new seeds.

Remark: a subtle mismatch

OOF predictions come from single fold models, test predictions from smoother averages; small for $K = 5$–10.

4 Weighted Ensembles

Motivation. Similar models often err differently, so weighted averages on held-out predictions help.

Example: three candidates

	linear	boosting	MLP
prediction	0.30	0.75	0.60
weight	0.10	0.60	0.30

$$\hat{y} = 0.10(0.30) + 0.60(0.75) + 0.30(0.60) = 0.66.$$

$\hat{y} = \sum_{m=1}^{M} w_m h_m(x)$ with fixed weights chosen on OOF performance.

Algorithm: greedy ensemble selection (Caruana et al.)

Start from the empty ensemble. Repeat R times: add (with replacement) the candidate whose inclusion gives the best OOF score of the *averaged* ensemble. The weight of model m is (number of times selected)$/R$.

Weights are convex; useless models get 0. This overfits OOF less than regressing y on z.

Theorem: ambiguity decomposition

For weights $w_m \geq 0$, $\sum_m w_m = 1$, and $\hat{y} = \sum_m w_m h_m$, under squared loss

$$(y - \hat{y})^2 = \sum_m w_m (y - h_m)^2 - \sum_m w_m (h_m - \hat{y})^2.$$

Hence the ensemble's error is never larger than the weighted average of the members' errors, and the improvement equals the weighted spread ("ambiguity") of the members' predictions.

Proof [Supplement]. $\sum_m w_m(y-h_m)^2 = \sum_m w_m\{(y-\hat{y})+(\hat{y}-h_m)\}^2 = (y-\hat{y})^2 + 2(y-\hat{y})\sum_m w_m(\hat{y}-h_m) + \sum_m w_m(\hat{y}-h_m)^2$, and the middle sum is $\hat{y}-\hat{y}=0$. ■

Gains need good *and* disagreeing members. For any convex loss Jensen gives the inequality (Exercise 7.3).

5 How the Three Mechanisms Fit Together in AutoGluon

	Fold bagging	Stacking	Weighted ensemble
learning	same model on K folds	higher model g on candidates' OOF predictions z and original features x	repeatedly selects candidates that improve internal validation
new case	average of fold predictions	feeds candidates' predictions and original features to $g(x,z)$	sums candidates' predictions with fixed weights
prediction	$\frac{1}{K}\sum_{k=1}^{K} h^{(-k)}(x)$	$g(x,z)$	$\sum_{m=1}^{M} w_m h_m(x)$
core role	less prediction variance, OOF generation	learns new prediction rules from candidates' error patterns	combines the selected candidates into one final prediction

With `num_stack_levels`, level-2 models are fold-bagged too and a final weighted ensemble combines the top layer; x is passed to every level.

6 Setting the Search Space

```
predictor = TabularPredictor(label=label, eval_metric=metric)
predictor.fit(train_data, presets=..., time_limit=..., hyperparameters=...)
```

Setting	Role
`eval_metric`	metric used to rank candidates and to select the ensemble
`presets`	bundles the candidate portfolio, ensembling, refitting and cost settings (e.g. quality versus speed)
`time_limit`	approximate time budget for `fit()`
`hyperparameters`	the candidate portfolio: which model families and which settings of each
`num_bag_folds`	number of folds used to create OOF predictions
`num_stack_levels`	number of extra stacking layers that use lower models' predictions as inputs

Match the metric to the decision (Ch. 1); report the time limit with the leaderboard.

Summary

- No single tabular model is always best; combining candidates is a decision in its own right.
- Stacking needs out-of-fold predictions; in-sample predictions reward memorization.
- Fold bagging creates OOF predictions and averages fold models for new cases; stacking learns $g(x,z)$; the weighted ensemble combines selected candidates with fixed convex weights.
- A convex combination's loss is at most the average member loss; gains come from diverse, individually strong candidates.

- AutoGluon manages fitting, validation and combination; the validation design and the final test remain the analyst's responsibility.

7 Selected exercises

Motivation. Best two-model blend, winner's optimism, and log loss under convex combination.

Exercise 7.1 (The best blend of two models)

Let $e_m = Y - h_m(X)$, $m = 1, 2$, with $s_m^2 = \mathbb{E}e_m^2$, $c = \mathbb{E}(e_1 e_2)$ and $\Delta = s_1^2 + s_2^2 - 2c = \mathbb{E}(e_1 - e_2)^2 > 0$. For $w \in \mathbb{R}$ let $h_w = wh_1 + (1-w)h_2$.
(a) Show that $\mathbb{E}\{Y - h_w(X)\}^2$ is minimized at $w^\star = (s_2^2 - c)/\Delta$, with minimum $M = (s_1^2 s_2^2 - c^2)/\Delta$.
(b) Show that $M \le \min(s_1^2, s_2^2)$, with equality iff $c = \min(s_1^2, s_2^2)$.
(c) Show that $w^\star \in [0, 1]$ iff $c \le \min(s_1^2, s_2^2)$.

Solution [Supplement].

(a) $Y - h_w = we_1 + (1-w)e_2$, so the risk is

$$q(w) = w^2 s_1^2 + 2w(1-w)c + (1-w)^2 s_2^2 = \Delta w^2 - 2(s_2^2 - c)w + s_2^2,$$

with leading coefficient $\Delta > 0$, minimized at $w^\star = (s_2^2 - c)/\Delta$:

$$q(w^\star) = s_2^2 - \frac{(s_2^2 - c)^2}{\Delta} = \frac{s_2^2(s_1^2 + s_2^2 - 2c) - (s_2^2 - c)^2}{\Delta} = \frac{s_1^2 s_2^2 - c^2}{\Delta}.$$

(b) Suppose $s_1^2 \le s_2^2$ (the other case is symmetric). Then

$$s_1^2 - M = \frac{s_1^2(s_1^2 + s_2^2 - 2c) - s_1^2 s_2^2 + c^2}{\Delta} = \frac{(s_1^2 - c)^2}{\Delta} \ge 0,$$

with equality iff $c = s_1^2$.

(c) $w^\star \ge 0$ iff $c \le s_2^2$, and $w^\star \le 1$ iff $s_2^2 - c \le \Delta$ iff $c \le s_1^2$.

For $c > \min s_m^2$ the optimal weight extrapolates; convex weights trade this for stability. ■

Exercise 7.2 (The winner's internal score is optimistic)

Candidates $k = 1, \dots, K$ have true risks R_k and internal validation estimates $\widehat{R}_k$ with $\mathbb{E}\widehat{R}_k = R_k$. Let $\hat{k} = \arg\min_k \widehat{R}_k$.
(a) Show that $\mathbb{E}\widehat{R}_{\hat{k}} \le \min_k R_k \le \mathbb{E}R_{\hat{k}}$.
(b) If $R_1 = \cdots = R_K = R$ and $\widehat{R}_k = R + \tau Z_k$ with $Z_k \overset{\text{iid}}{\sim} N(0,1)$, show that the optimism is $R - \mathbb{E}\widehat{R}_{\hat{k}} = \tau\, \mathbb{E}\max_k Z_k$, and that it equals $\tau/\sqrt{\pi}$ for $K = 2$ and does not decrease in K.

Solution [Supplement].

(a) $\widehat{R}_{\hat{k}} \le \widehat{R}_j \ \forall j \Rightarrow \mathbb{E}\widehat{R}_{\hat{k}} \le R_j$; minimize over j. And $R_{\hat{k}} \ge \min_k R_k$ pointwise.

(b)

Step 1. $\min_k(R + \tau Z_k) = R - \tau \max_k(-Z_k)$ and $(-Z_k) \overset{d}{=} (Z_k)$.

Step 2. For $K = 2$, $\max(Z_1, Z_2) = \frac{1}{2}(Z_1 + Z_2) + \frac{1}{2}|Z_1 - Z_2|$ and $Z_1 - Z_2 \sim N(0, 2)$, so

$$\mathbb{E}\max(Z_1, Z_2) = \tfrac{1}{2}\mathbb{E}|Z_1 - Z_2| = \tfrac{1}{2}\sqrt{2}\sqrt{2/\pi} = 1/\sqrt{\pi} \approx 0.56.$$

Step 3. An extra candidate can only raise the maximum.

Hence the final number must come from a test set not used for selection. ■

Exercise 7.3 (Convex combinations under log loss)

Let $\ell(y,p) = -y\log p-(1-y)\log(1-p)$ for $y \in \{0,1\}$, $p \in (0,1)$, members $p_1,\dots,p_M \in (0,1)$, weights $w_m \geq 0$ with $\sum_m w_m = 1$, and $\bar{p} = \sum_m w_m p_m$.
(a) Show that $\ell(y,\bar{p}) \leq \sum_m w_m \ell(y,p_m)$, with equality iff all p_m with $w_m > 0$ are equal.
(b) Two cases, both with $y = 1$; member 1 predicts $(0.9, 0.1)$ and member 2 predicts $(0.1, 0.9)$. Compare the average log losses of the members and of their equal-weight average.

Solution [Supplement].

(a)

Step 1. $(-\log p)'' = 1/p^2 > 0$, $(-\log(1-p))'' = 1/(1-p)^2 > 0$: strictly convex.

Step 2. Strict Jensen gives the inequality, with equality iff the p_m with $w_m > 0$ coincide.

Summing over cases, the same holds for average log loss on any data set.

(b) Each member: $\frac{1}{2}(-\log 0.9 - \log 0.1) \approx \frac{1}{2}(0.105 + 2.303) = 1.204$. The average predicts 0.5: $\log 2 \approx 0.693$, beating *both*. ■